Dimensionality Reduction

PCA & t-SNE

Machine Learning with Python — On-site session (Oct 23)

EUGLOH — *Problem Solving Using Open-Source Languages; R and Python*

University of Novi Sad

Today's session

- **Why** reduce dimensions at all?
- **PCA** — finding the directions where the data varies most
- **t-SNE** — drawing a map that keeps neighbours together
- How to **read** these plots — and how to **avoid fooling yourself**
- How both fit into the pipeline with the models you already know

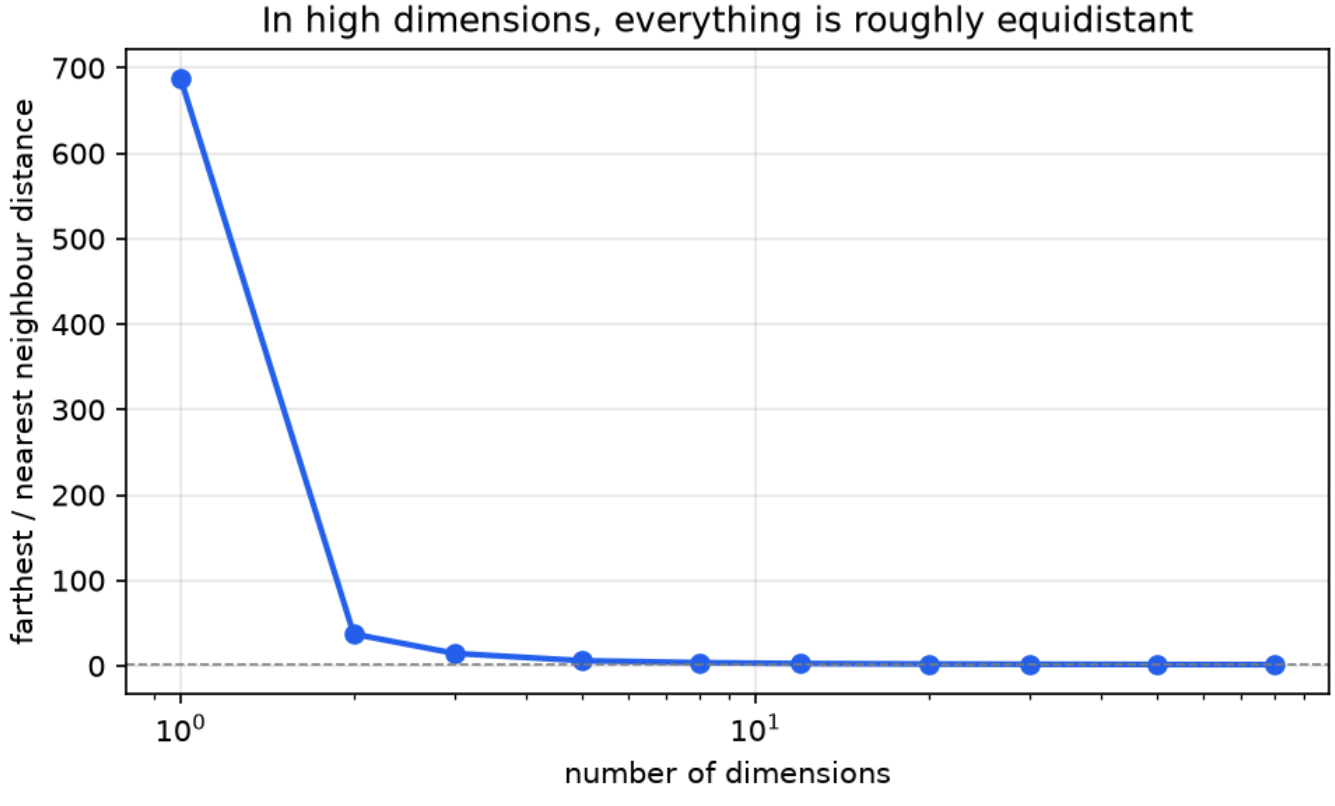
On-site session — the first of two today.

Part 1 — Why reduce dimensions?

Real data has **many features**: a tiny 8×8 image is already 64 numbers, a small image at 224×224 is **150 528**. We rarely need all of them at once.

- **Visualisation** — we cannot draw 64 dimensions; project to 2 so our eyes can help.
- **The curse of dimensionality** — in high dimensions data becomes *sparse* and distances stop being informative.
- **Compression & denoising** — keep the signal, drop the noise.
- **Speed** — fewer features mean faster training and prediction.

The goal: keep the **structure**, throw away the **redundancy**.


In high dimensions, everything is roughly equidistant
farthest / nearest neighbour distance
700
600
500
400
300
200
100
0
10^0
10^1
number of dimensions

Part 2 — PCA

The idea: follow the variance

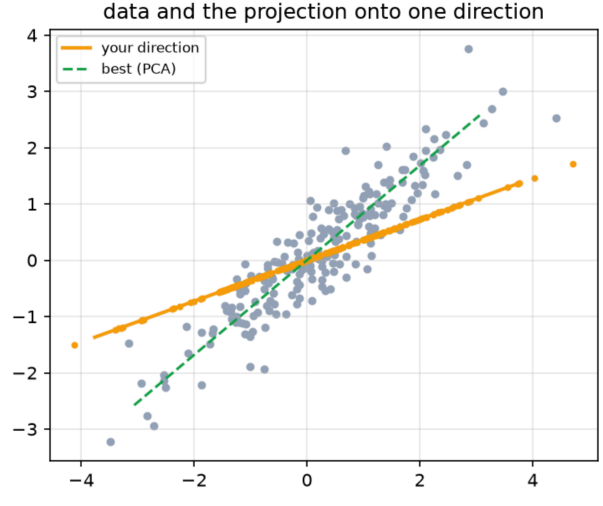
PCA looks for the direction along which the data **varies the most**, then the next-most, perpendicular to the first, and so on.

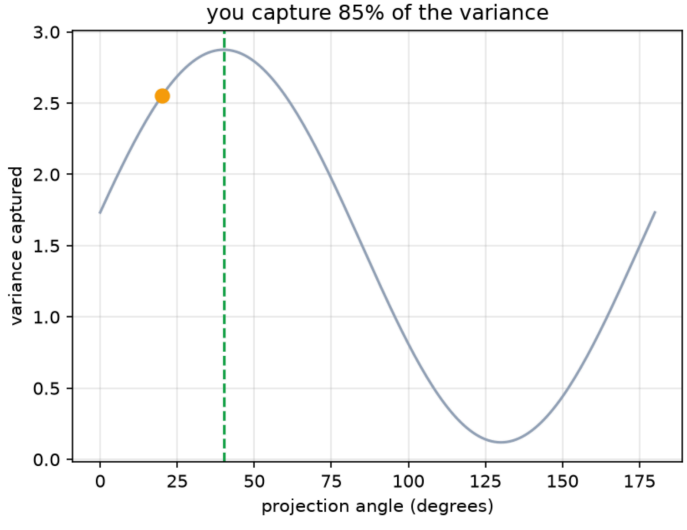
- Projecting onto those directions keeps as much **spread** as possible.
- Two correlated features carry mostly the *same* information — PCA replaces them with one direction and loses almost nothing.
- In plain words: **rotate the axes** so they line up with how the data is stretched, then drop the squished directions.

PCA — find the best direction

The grey cloud below is real data. Drag the angle to choose a direction to project onto (orange dots are the projections). The right panel shows how much **variance** that direction captures. Which angle wins?

projection angle (degrees) 20



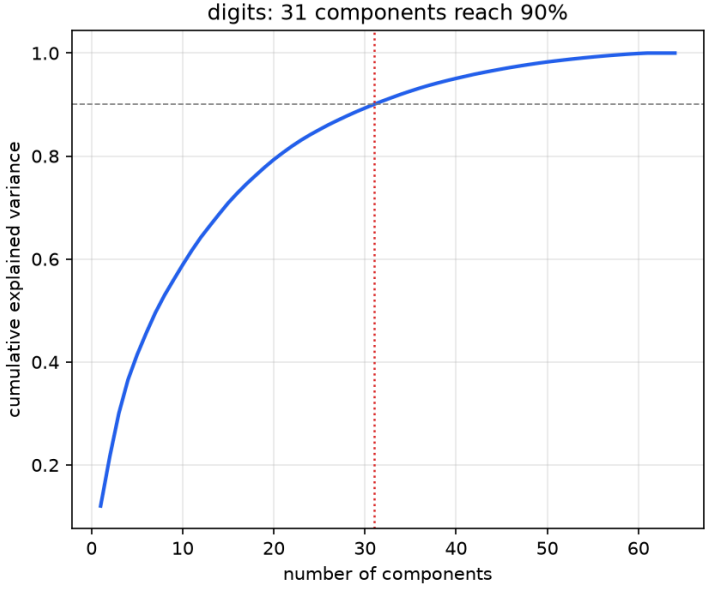


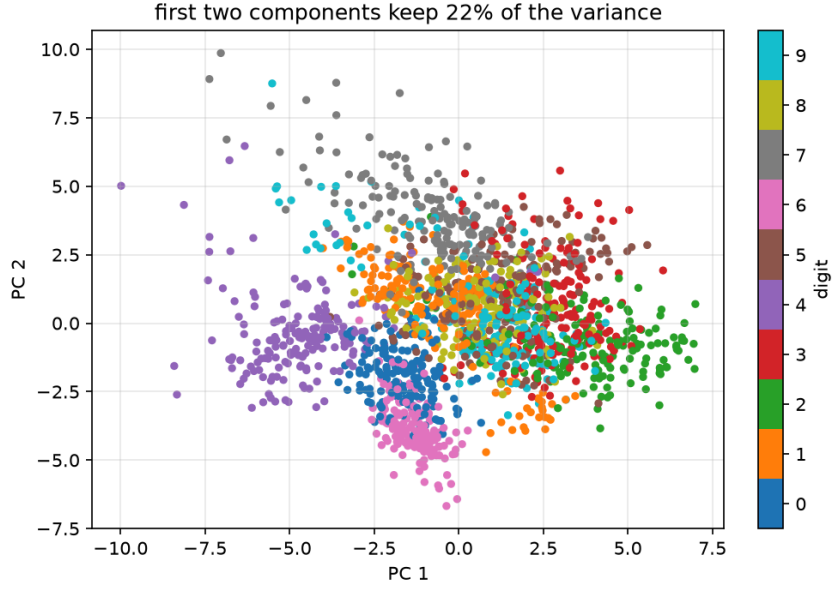
PCA — the recipe

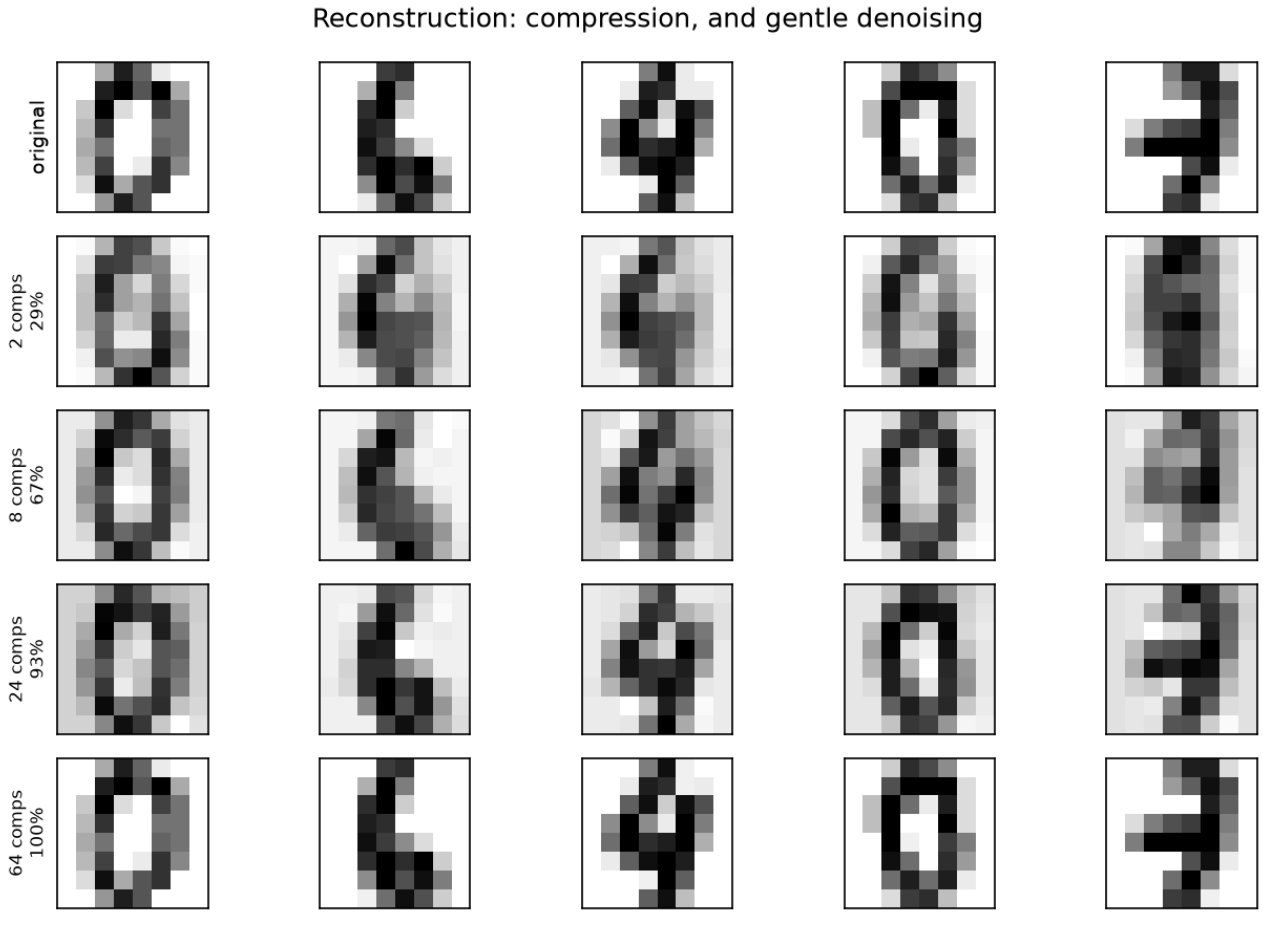
1. **Centre** the data: subtract the mean of each feature.
2. **Find the directions of maximum variance** — the eigenvectors of the covariance matrix (equivalently, the **SVD** of the centred data).
3. **Project**: $Z = XW$, where the columns of W are the top k directions.
4. Each direction captures a **variance** (its eigenvalue) — sum them to see how much you kept.

In plain words: **centre, find the stretch directions, keep the big ones.**

Two things to remember: always **standardise** your features first (PCA is scale-sensitive), and the components are **uncorrelated** by construction.


digits: 31 components reach 90%
cumulative explained variance
number of components
1.0
0.8
0.6
0.4
0.2
0
10
20
30
40
50
60


first two components keep 22% of the variance
PC 2
PC 1
digit
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
9
8
7
6
5
4
3
2
1
0


Reconstruction: compression, and gentle denoising
original
2 comps
29%
8 comps
67%
24 comps
93%
64 comps
100%

PCA — what to watch out for

- **It is linear.** Curved or clustered structure is not captured; if the data lies on a curve, PCA will flatten it.
- **Scale matters.** An unscaled feature with a large range dominates. Standardise first.
- **Maximum variance ≠ best separation.** PCA does not look at the labels. A direction with little variance can be the one that separates the classes.
- **Components are hard to name.** Each is a mix of all original features, so interpretability drops.

Still the best **first** thing to try: fast, deterministic, and a good sanity check before fancier methods.

Part 3 — t-SNE

The idea: keep neighbours together

t-SNE does not care about variance. It asks a **neighbourhood** question: *which points are close to which, in the original space?* — then lays them out in 2-D so that the same points stay close.

- High-dimensional similarities become **probabilities** (a Gaussian around each point).
- Low-dimensional similarities do the same with a heavier-tailed distribution.
- It then nudges the points to make the two sets of probabilities match (minimising a **KL divergence**).

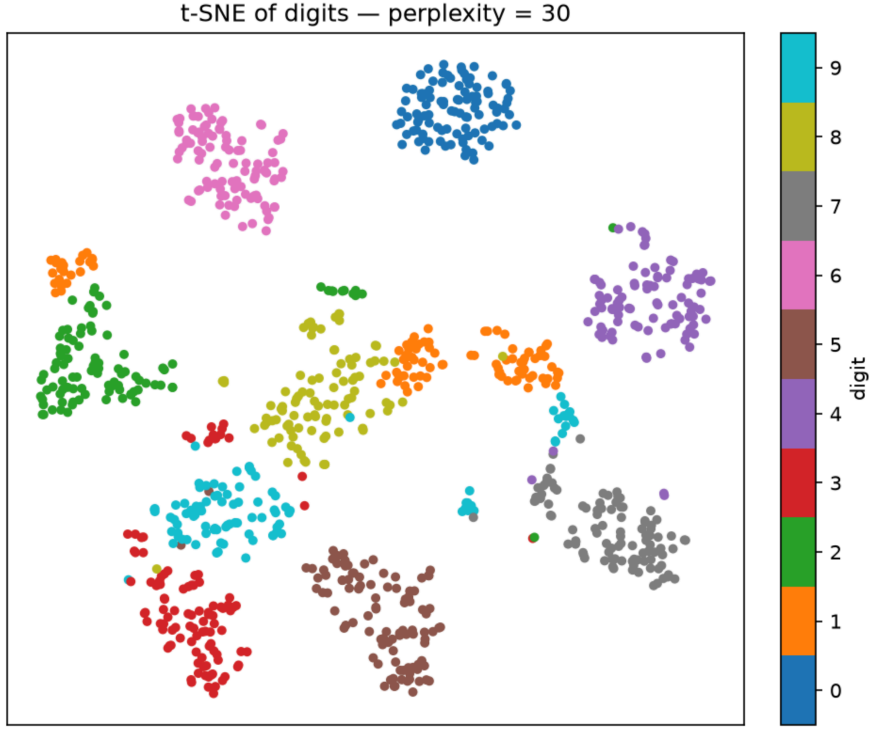
Perplexity controls how many neighbours each point cares about — roughly the "attention span" of the map.

In plain words: **attract neighbours, push strangers apart.**

t-SNE — the role of perplexity

The same digits data, mapped to 2-D with different perplexities. Low values focus on very local structure (many tiny islands); high values blend neighbours together (fewer, larger clumps). There is no single "correct" value — look for a map that is **stable across a few settings**.

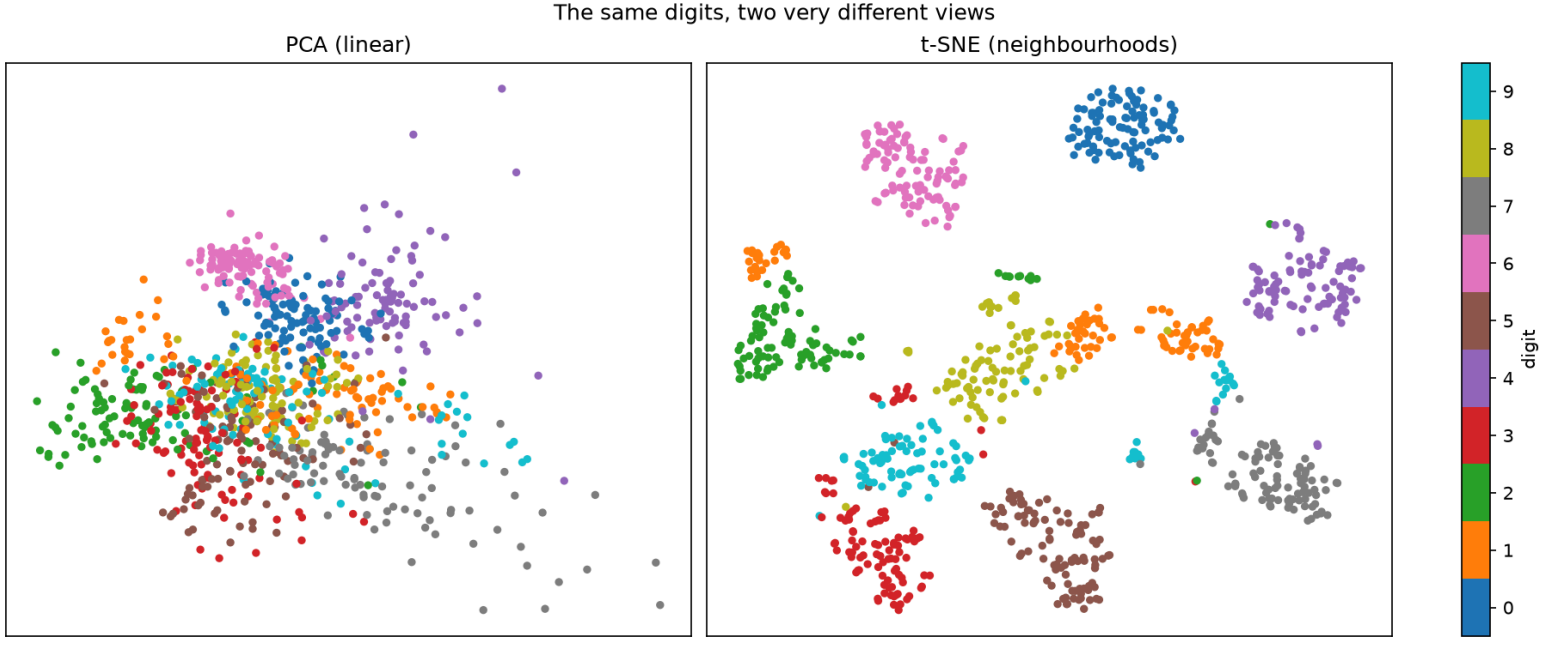
perplexity 30



t-SNE — how to read it (and not fool yourself)

- **Cluster distances are not meaningful.** Two blobs far apart are not "more different" than two blobs close together.
- **Cluster sizes are not meaningful** either — t-SNE expands dense clusters and squeezes sparse ones.
- **It is stochastic.** Run it a few times; trust what is stable.
- **Hyperparameters matter.** Perplexity (and the learning rate) change the picture; do not over-interpret one run.
- **Do not feed the 2-D coordinates into a classifier.** It is built for *looking*, not for prediction (use PCA or the raw features for that).

t-SNE is a **microscope for local structure**, not a map of global distances.


The same digits, two very different views
PCA (linear)
t-SNE (neighbourhoods)
digit
9
8
7
6
5
4
3
2
1
0

Summary

- **PCA** finds the directions of **maximum variance** by rotating the axes; it is **linear, fast and deterministic**, and great for compression, denoising and a first look.
- **t-SNE** builds a 2-D **neighbourhood map**; it is the better tool for *seeing* clusters, but its distances and sizes are **not** meaningful and it must not be used as input features.
- **Standardise first**, and remember **variance is not separation**.
- Choose the tool for the **job**: PCA to reduce/compress/feed a model, t-SNE to look.

Where to go next

- **Next lecture: pretrained models** — using powerful feature extractors, and how combining them with PCA/t-SNE and the supervised models from earlier lets us visualise and classify real image data.
- We will apply all of this to the **course dataset** in the hands-on part.